TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS:

JONATHAN ENRIQUE YUGCHA

PARALELO:

"A"

SEMESTRE:

QUINTO

ACTIVIDAD:

Actividad 1

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad _1_

1. Introducción

El rescate y la adopción de animales requieren una organización eficiente para registrar los casos reportados, controlar el estado de los animales rescatados, asignar responsables y facilitar los procesos de atención veterinaria y adopción.

En muchos refugios, estos procedimientos pueden gestionarse mediante registros manuales, hojas de cálculo o comunicaciones informales, lo que dificulta el seguimiento de cada caso y la coordinación entre ciudadanos, voluntarios, veterinarios y administradores.

Por esta razón, se propone desarrollar un Sistema de Control de Rescate y Adopción Animal, una solución informática que permita centralizar la información de los animales, registrar sus necesidades, asignar responsables y mantener un historial de las actividades realizadas.

El diseño se fundamentará en principios de programación orientada a objetos, principios SOLID y patrones de diseño de software. Además, se utilizará GitHub con la metodología Gitflow para organizar el desarrollo, controlar las versiones y facilitar la incorporación de nuevas funcionalidades

2. Objetivos

Objetivo general

Diseñar la arquitectura lógica de un sistema de gestión de rescate y adopción animal mediante la aplicación de patrones de diseño de software, diagramas UML y el flujo de trabajo Gitflow, con la finalidad de construir una solución escalable, reutilizable y mantenible.

Objetivos específicos

- Identificar los actores involucrados y los requerimientos funcionales y no funcionales del sistema.
- Diseñar diagramas UML que representen las interacciones, clases y secuencias principales de la aplicación.

- Aplicar patrones de diseño como MVC, Factory Method, Observer y Singleton para organizar las responsabilidades del sistema.
- Establecer una arquitectura lógica basada en la separación de responsabilidades y los principios SOLID.
- Configurar un repositorio público en GitHub utilizando Gitflow para gestionar ramas, funcionalidades y versiones.
- Documentar las decisiones técnicas y justificar los patrones de diseño seleccionados.

Actores principales

El sistema tendrá cuatro actores principales.

Actor	Descripción	Responsabilidades
Ciudadano o reportante	Persona que identifica un animal que necesita ayuda.	Registrar un reporte de rescate y consultar el estado de su reporte.
Voluntario	Persona encargada de colaborar en las actividades de rescate y cuidado.	Consultar casos asignados, actualizar avances y registrar actividades.
Veterinario	Profesional responsable de la atención médica.	Evaluar animales, registrar tratamientos y actualizar su estado clínico.
Administrador del refugio	Persona encargada de supervisar las operaciones.	Gestionar casos, asignar responsables, administrar expedientes y supervisar adopciones.

Requerimientos funcionales:

Los requerimientos funcionales describen las operaciones que el sistema debe permitir.

- **Registrar reportes:** Permitir que un ciudadano registre un caso de rescate.
- **Categorizar especies:** Clasificar los animales como perros, gatos o fauna silvestre
- **Crear expedientes:** Generar un expediente con los datos de cada animal rescatado

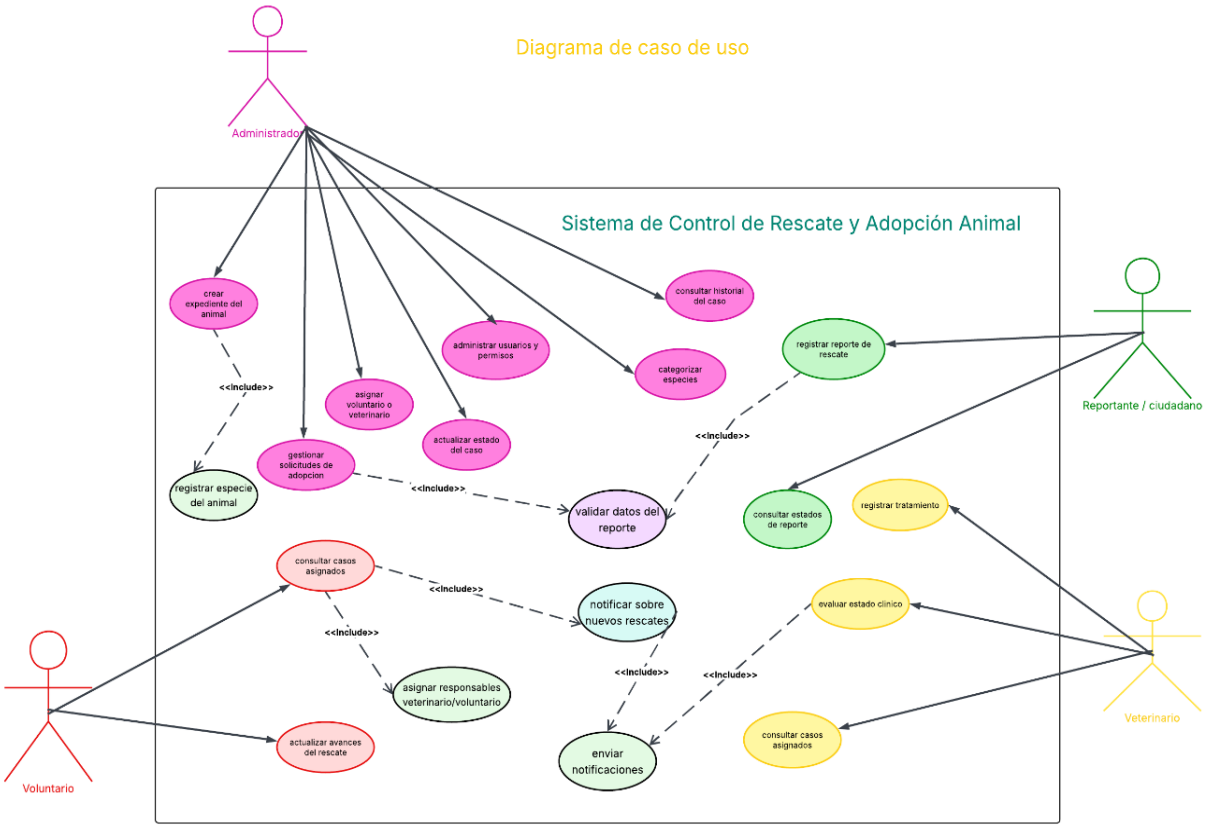
- **Asignar responsables:** Permitir al administrador asignar voluntarios o veterinarios.
- **Gestionar estados:** Actualizar el estado del caso según su evolución
- **Registrar tratamientos:** Guardar evaluaciones veterinarias y tratamientos realizados.
- **Notificar responsables:** Enviar notificaciones cuando se registren casos nuevos o se produzcan cambios importantes
- **Gestionar adopciones:** Registrar solicitudes, evaluar candidatos y formalizar adopciones
- **Consultar expedientes:** Buscar animales por identificador, especie o estado.
- **Consultar historial:** Mantener un registro de las modificaciones y actividades realizadas.
- **Administrar usuarios:** Gestionar las cuentas, roles y permisos del personal
- **Controlar disponibilidad:** Identificar animales disponibles para adopción

Requerimientos No Funcionales

- **Escalabilidad:** Permitir incorporar nuevos tipos de animales y funcionalidades
- **Mantenibilidad:** Organizar el código en módulos con responsabilidades claramente definidas.
- **Seguridad:** Controlar el acceso mediante autenticación y autorización por roles.
- **Trazabilidad:** Registrar los cambios del sistema y mantener un historial de versiones.
- **Usabilidad:** Ofrecer una interfaz clara y fácil de utilizar.
- **Fiabilidad:** Validar la información para evitar registros inconsistentes
- **Reutilización:** Utilizar componentes y patrones que puedan aprovecharse en diferentes módulos.
- **Integridad de datos:** Evitar que los cambios de estado produzcan información contradictoria.

Diagrama de caso de uso

Crear un diagrama de casos de uso para entender las interacciones del personal con la plataforma.

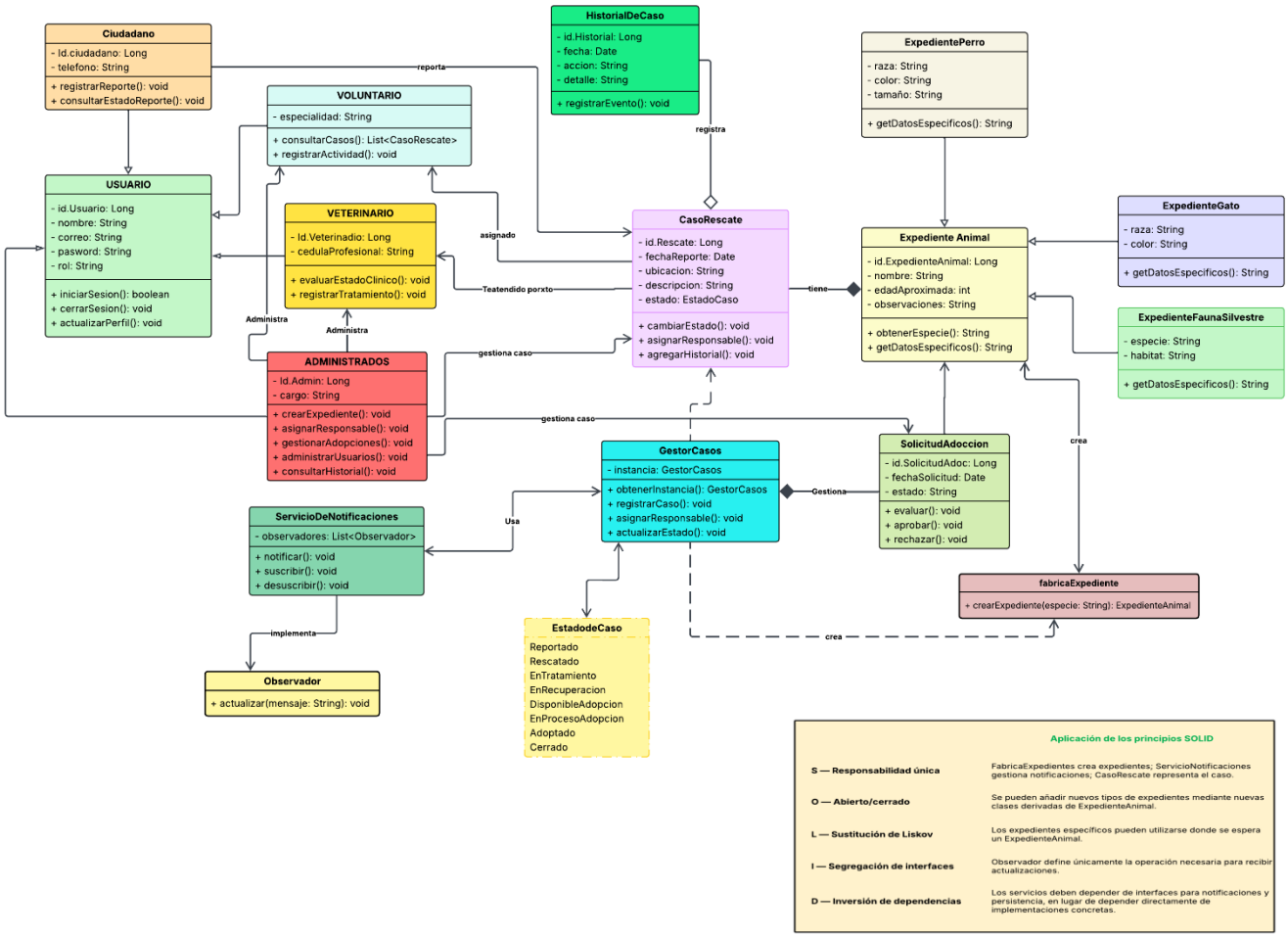


Este diagrama muestra las operaciones disponibles para los diferentes actores, el ciudadano puede registrar reportes y consultar su estado. El voluntario y el veterinario trabajan con los casos que tienen asignados, mientras que el administrador supervisa los expedientes, las asignaciones y las adopciones.

Diagrama de clases UML

El diagrama de clases representa la estructura de los objetos, sus atributos, operaciones y relaciones.

DIAGRAMA DE CLACE UML



El diagrama de clases UML representa la estructura estática del Sistema de Control de Rescate y Adopción Animal, identificando las clases principales, sus atributos, métodos y relaciones. La herencia permite organizar los diferentes tipos de usuarios y expedientes, mientras que las asociaciones representan las relaciones entre los casos de rescate, los responsables, el historial y las solicitudes de adopción. Asimismo, la fábrica de expedientes centraliza la creación de objetos según la especie, y el servicio de notificaciones permite implementar el patrón Observer. Finalmente, la separación de responsabilidades y el uso de abstracciones contribuyen a aplicar los principios SOLID, favoreciendo la mantenibilidad y extensibilidad del sistema.

Descripción de la arquitectura

El Sistema de Control de Rescate y Adopción Animal se diseñará utilizando una arquitectura por capas basada en el patrón MVC (Modelo-Vista-Controlador). Esta arquitectura permitirá separar la interfaz de usuario, la lógica del sistema y el acceso a los datos, facilitando el mantenimiento, las pruebas y la incorporación de nuevas funcionalidades.

El sistema permitirá registrar animales rescatados, asignar voluntarios y veterinarios, administrar expedientes, registrar tratamientos, gestionar solicitudes de adopción y enviar notificaciones automáticas cuando se registre un nuevo rescate.

Se propone utilizar las siguientes tecnologías:

- **Frontend:** Angular o HTML, CSS y JavaScript.
- **Backend:** Java con Spring Boot.
- **Base de datos:** PostgreSQL.
- **Control de versiones:** Git y GitHub.
- **Diagramas UML:** Lucidchart o draw.io

Aplicación de patrones de diseño

Se utilizarán cuatro patrones de diseño: Singleton, Observer, Factory Method y MVC. Cada uno resolverá una necesidad específica del sistema.

1. **Patrón Singleton**

Propósito: garantizar que una clase tenga una única instancia dentro del contexto definido para esa aplicación.

Aplicación en el sistema: puede utilizarse para un componente de configuración o para un administrador central de recursos que realmente requiera una única instancia.

Por ejemplo, el sistema necesita coordinar el registro de rescates y mantener reglas comunes para las operaciones. Sin embargo, no es necesario convertir todas las clases de gestión en Singleton.

En Java con Spring Boot, los servicios y repositorios suelen utilizarse como beans Singleton por defecto. Por eso, no es recomendable crear manualmente un Singleton

para cada servicio ni para cada conexión a la base de datos. La conexión debe gestionarse mediante un *pool* de conexiones.

Problema que resuelve: evita crear varias instancias independientes de un componente que debe tener un estado o una configuración compartidos.

2. **Patrón Observer**

Propósito: establecer una relación de suscripción mediante la cual varios objetos reciben notificaciones cuando ocurre un evento de interés.

Aplicación en el sistema: cuando se registra un nuevo rescate, los voluntarios suscritos a las notificaciones reciben un aviso sobre el nuevo caso.

Problema que resuelve: evita que la lógica de registro de rescates tenga que conocer directamente a cada voluntario o implementar por separado el envío de cada notificación

Flujo de funcionamiento

- El ciudadano registra un rescate.
- El sistema guarda el caso en la base de datos.
- Se publica el evento Nuevo RescateRegistrado.
- El componente de notificaciones recibe el evento.
- Los destinatarios suscritos reciben el aviso mediante el canal configurado.

3. **Patrón Factory Method**

Propósito: delegar la creación de objetos a un método de fábrica, evitando que el resto del sistema dependa directamente de las clases concretas que necesita instanciar.

Aplicación en el sistema: crear expedientes distintos según la especie del animal rescatado.

El sistema deberá manejar, como mínimo:

- Expediente de perro.
- Expediente de gato.
- Expediente de fauna silvestre.

Problema que resuelve: permite incorporar nuevos tipos de expedientes sin tener que modificar todas las partes de la aplicación que crean o utilizan esos objetos.

4. **Patrón MVC (Modelo-Vista-Controlador)**

Propósito: separar los datos y las reglas del sistema, la interfaz gráfica y el procesamiento de las solicitudes del usuario.

Problema que resuelve: evita mezclar la presentación, las reglas de negocio y el acceso a los datos en una sola clase.

Justificación del uso de Gitflow

Gitflow es un modelo de trabajo basado en ramas de Git que permite organizar el desarrollo de un proyecto, separar las funcionalidades en construcción de las versiones estables y administrar las correcciones.

En este sistema, Gitflow facilitará el trabajo sobre funcionalidades como el registro de rescates, la administración de expedientes, las notificaciones y las solicitudes de adopción sin modificar directamente la rama de producción.

3. Conclusiones

El diseño propuesto para el Sistema de Control de Rescate y Adopción Animal permite organizar de manera estructurada las actividades relacionadas con el registro de rescates, la atención veterinaria, la asignación de responsables y la gestión de adopciones.

El análisis de requerimientos y la identificación de actores establecen las bases para comprender las necesidades de los ciudadanos y del personal del refugio. Por su parte, los diagramas UML permiten representar las interacciones, las clases principales y los flujos de comunicación entre los componentes.

Asimismo, los principios SOLID favorecen la separación de responsabilidades, la reutilización del código y la incorporación de nuevas funcionalidades. La arquitectura por capas complementa estos principios al separar la presentación, la lógica de aplicación, el dominio y la infraestructura.

Configurar su identidad local



Iniciamos git en nuestra carpeta

```
WINDOWS 10 PRO@DESKTOP-OM2QIR7 UCRT64 ~/OneDrive/Documents/actividad1 Jonathan Yugcha/actividad1 Git
Hub (main)
$ git init
Reinitialized existing Git repository in C:/Users/WINDOWS 10 PRO/OneDrive/Documents/actividad1 Jonat
han Yugcha/actividad1 GitHub/.git/

WINDOWS 10 PRO@DESKTOP-OM2QIR7 UCRT64 ~/OneDrive/Documents/actividad1 Jonathan Yugcha/actividad1 Git
Hub (main)
$
```

Realizamos el primer commit

```
WINDOWS 10 PRO@DESKTOP-OM2QIR7 UCRT64 ~/OneDrive/Documents/actividad1 Jonathan Yugcha/actividad1 Git
Hub (main)
$ git commit -m "commit inicial: subida de la primera version de la presentación centro de atención
y rescate animal"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'WINDOWS 10 PRO@DESKTOP-OM2QIR7.(none)')

WINDOWS 10 PRO@DESKTOP-OM2QIR7 UCRT64 ~/OneDrive/Documents/actividad1 Jonathan Yugcha/actividad1 Git
Hub (main)
$
```

Subir a GitHub (La Nube)

Ya hemos creado el repositorio en GITHUB procedemos a realizar

Vincular la carpeta local con GitHub

```
WINDOWS 10 PRO@DESKTOP-OM2QIR7 UCRT64 /e/actividad1 Jonathan Yugcha/actividad1 GitHub
$ git remote add origin https://github.com/JonathanYugcha/Sistema-de-control-de-Rescate-y-Adopci-n-A
nimal.git
fatal: detected dubious ownership in repository at 'E:/actividad1 Jonathan Yugcha/actividad1 GitHub'
'E:/actividad1 Jonathan Yugcha/actividad1 GitHub' is on a file system that does not record ownership
To add an exception for this directory, call:

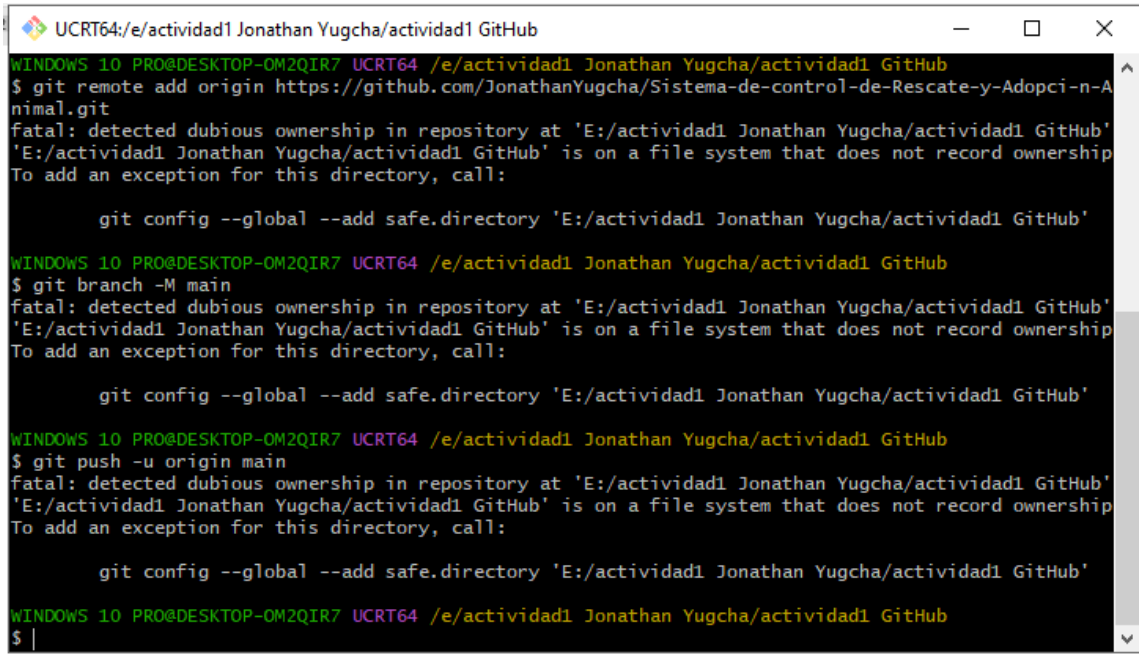
        git config --global --add safe.directory 'E:/actividad1 Jonathan Yugcha/actividad1 GitHub'

WINDOWS 10 PRO@DESKTOP-OM2QIR7 UCRT64 /e/actividad1 Jonathan Yugcha/actividad1 GitHub
$
```

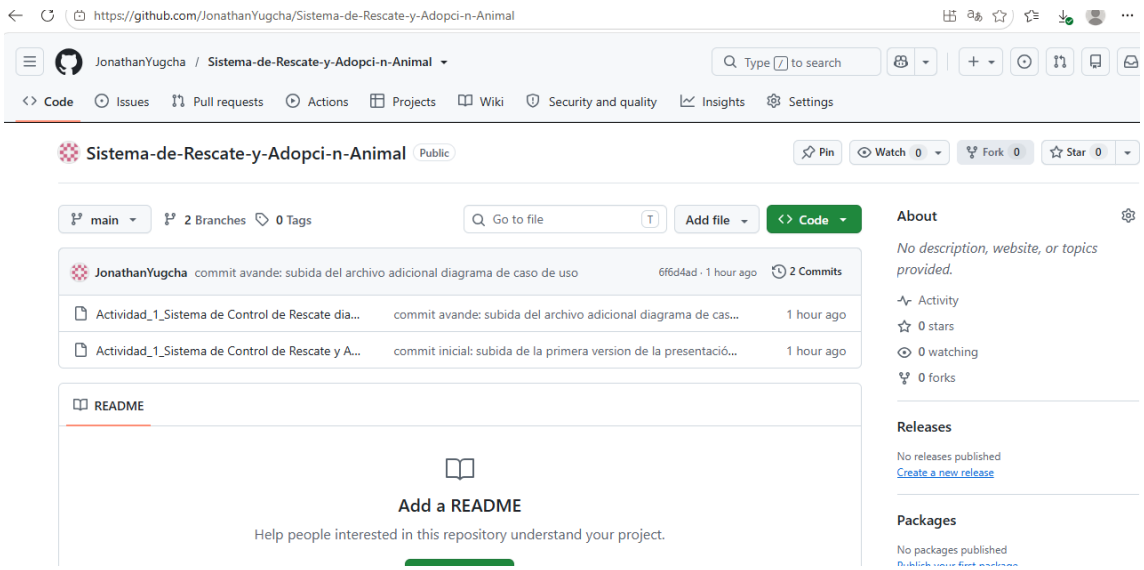
```
WINDOWS 10 PRO@DESKTOP-OM2QIR7 UCRT64 /e/actividad1 Jonathan Yugcha/actividad1 GitHub
$ git branch -M main
fatal: detected dubious ownership in repository at 'E:/actividad1 Jonathan Yugcha/actividad1 GitHub'
'E:/actividad1 Jonathan Yugcha/actividad1 GitHub' is on a file system that does not record ownership
To add an exception for this directory, call:

        git config --global --add safe.directory 'E:/actividad1 Jonathan Yugcha/actividad1 GitHub'

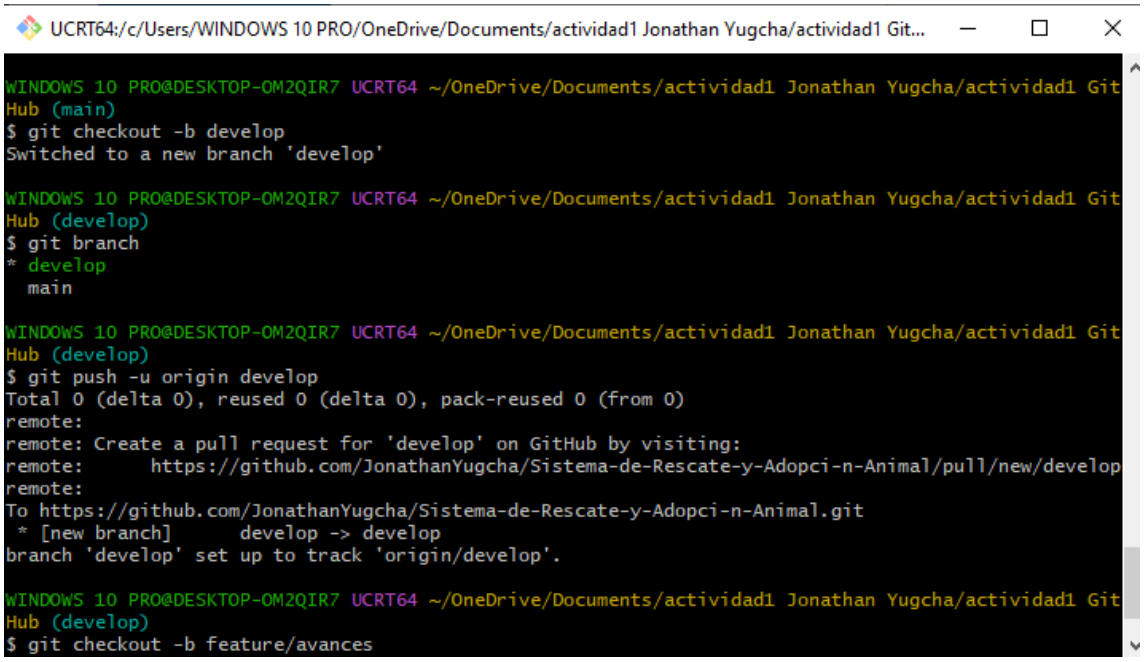
WINDOWS 10 PRO@DESKTOP-OM2QIR7 UCRT64 /e/actividad1 Jonathan Yugcha/actividad1 GitHub
$
```

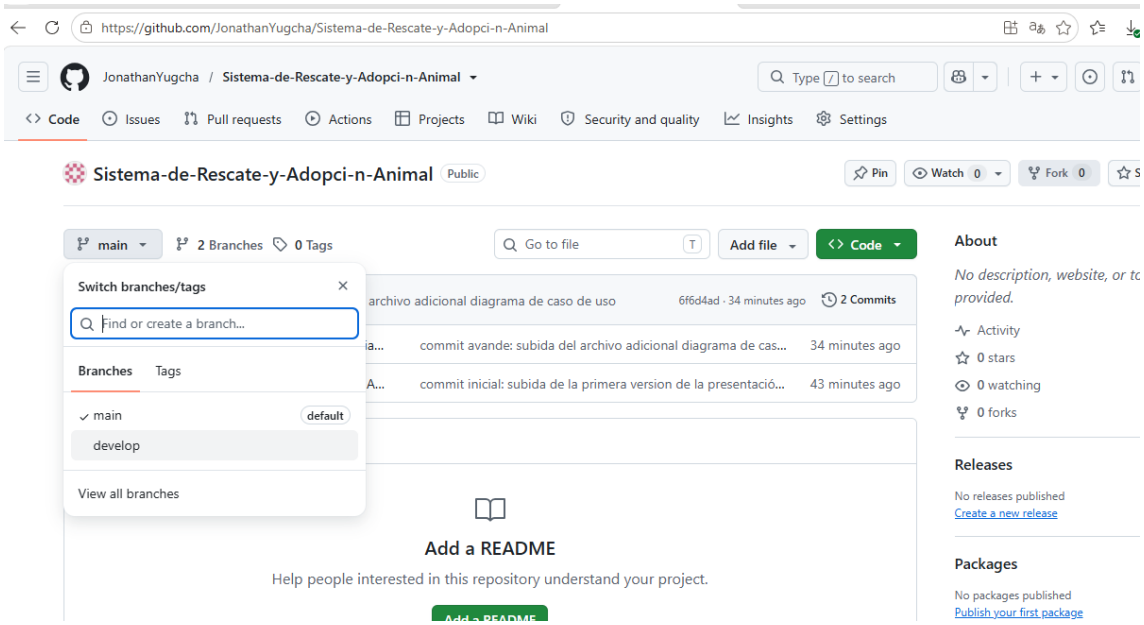


Tenemos lo commit en repositorio de github



Creamos la rama develop





Cremos la rama feature/avances

